

# Informatics II Exam FS 2025

June 5, 2025

|          |                                |           |
|----------|--------------------------------|-----------|
| <b>1</b> | <b>Task 1 (18 points)</b>      | <b>2</b>  |
| 1.1      | Task 1.1 (9 points) . . . . .  | 2         |
| 1.2      | Task 1.2 (9 points) . . . . .  | 3         |
| <b>2</b> | <b>Task 2 (19 points)</b>      | <b>4</b>  |
| 2.1      | Task 2.1 (8 points) . . . . .  | 5         |
| 2.2      | Task 2.2 (11 points) . . . . . | 6         |
| <b>3</b> | <b>Task 3 (26 points)</b>      | <b>7</b>  |
| 3.1      | Task 3.1 (10 points) . . . . . | 7         |
| 3.2      | Task 3.2 (12 points) . . . . . | 9         |
| 3.3      | Task 3.3 (4 points) . . . . .  | 11        |
| <b>4</b> | <b>Task 4 (27 points)</b>      | <b>12</b> |
| 4.1      | Task 4.1 (3 points) . . . . .  | 12        |
| 4.2      | Task 4.2 (10 points) . . . . . | 13        |
| 4.3      | Task 4.3 (14 points) . . . . . | 14        |

# 1 Task 1 (18 points)

## 1.1 Task 1.1 (9 points)

Consider an array  $A$  with  $n$  integer elements. Write a C function `void sort(int[] A, int n)` that sorts the integers of  $A$  in ascending order. Your solution may not use any additional composite data structures. Explain the time complexity of your solution.

solution 1 (selection sort):

**plan:** In each step exchange the leftmost element of the unsorted part with the smallest element of the unsorted part. This must be done  $n - 1$  times. The computational complexity is determined by the number of comparisons to find the smallest element. The total number of comparisons is  $\sum_{i=1}^{n-1} i = n(n-1)/2 = \Theta(n^2)$

**example:**

|   |   |   |   |   |   |                     |
|---|---|---|---|---|---|---------------------|
| 7 | 8 | 5 | 3 | 9 | 4 | $n - 1$ comparisons |
| 3 | 8 | 5 | 7 | 9 | 4 | $n - 2$ comparisons |
| 3 | 4 | 5 | 7 | 9 | 8 | ...                 |
| 3 | 4 | 5 | 7 | 9 | 8 | 2 comparisons       |
| 3 | 4 | 5 | 7 | 9 | 8 | 1 comparison        |
| 3 | 4 | 5 | 7 | 8 | 9 |                     |

**code:**

```
1 void selection_sort(int A[], int n) {
2     for (int i = 0; i < n - 1; i++) {
3         int min_index = i;
4         for (int j = i + 1; j < n; j++) {
5             if (A[j] < A[min_index]) { min_index = j; }
6         }
7         swap(A, i, min_index);
8     }
9 }
```

## 1.2 Task 1.2 (9 points)

Consider an **array**  $A$  with  $n$  integer elements. All integers of  $A$  have a value between 0 and  $k - 1$ . Assume  $k < n$ . A **frequency array**  $F$  for  $A$  is an array of length  $k$  in which element  $F[i]$  is equal to the number of times that  $i$  appears in  $A$ .

- a) Consider array  $A = [2, 0, 2, 2, 2]$  with  $n = 5$  and  $k = 4$ . Draw the frequency array  $F$  for  $A$ . Clearly work out the indices and values of  $F$ .

$$F = \begin{array}{|c|c|c|c|} \hline 1 & 0 & 4 & 0 \\ \hline 0 & 1 & 2 & 3 \\ \hline \end{array}$$

- b) Write a C function `void sortK(int[] A, int n, int k)` that sorts  $A$  in ascending order. The time complexity of your solution must be  $O(n)$  and as part of your solution you are allowed to define and use a frequency array for  $A$ . No additional composite data structures may be used.

**plan:** the first step is to construct a frequency array by going through the elements of  $A$ . initially the frequency of all elements in  $F$  is zero. for each element in  $A$  the frequency of this element in  $F$  is increased. the second step overwrites  $A$  by going through  $F$  and adding  $F[i]$  copies of  $i$  to  $A$ .

**example:** the initial array  $A$  is  $[2, 0, 2, 2, 2]$ ; the frequency array  $F$  for  $A$  is  $[1, 0, 4, 0]$ ; starting from position 0 we write into  $A$  the following elements: 1 zero, 0 ones, 4 twos and 0 threes; the resulting array  $A$  is  $[0, 2, 2, 2, 2]$ .

**code:**

```
1 void sortK(int A[], int n, int k) {
2     int F[k];
3     for (int i = 0; i < k; i++) { F[i] = 0; }
4     for (int i = 0; i < n; i++) { F[A[i]] += 1; }
5     int x = 0;
6     for (int i = 0; i <= k - 1; i++) {
7         for (int j = 0; j < F[i]; j++) {
8             A[x] = i;
9             x++;
10        }
11    }
12 }
```

## 2 Task 2 (19 points)

For this task we consider a hash table HT with  $m$  slots that uses chaining to resolve collisions. The following code listing shows a C code fragment with declarations and definitions for the task.

```
1  #define m 100
2
3  struct node { int value; struct node* next; };
4  struct node* HT[m];
5
6  int h(int k) { return k % m; }
7
8  void HTinsert(int k) { ... };
9
10 void print_overlap(int A1[], int A2[], int n) { ... };
```

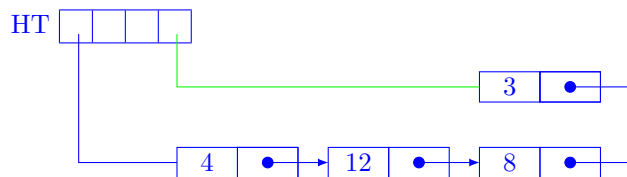
## 2.1 Task 2.1 (8 points)

Provide the complete C code for procedure `void HTInsert(int k)`. The procedure inserts value  $k$  into hash table HT.

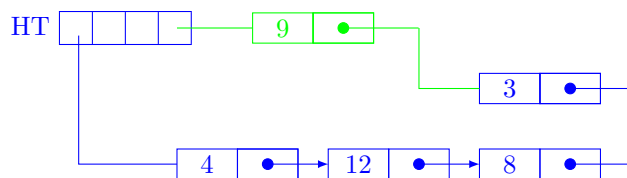
**plan:** allocate space for the new element and add it at the beginning of the linked list for this element.

**example:**

hash table with elements 3, 4, 8, 12 ( $m = 4$ ):



hash table after inserting 9:



**code:**

```
1 void HTInsert(int k) {
2     struct node *n = malloc(sizeof(struct node));
3     n->value = k;
4     n->next = HT[h(k)];
5     HT[h(k)] = n;
6 }
```

## 2.2 Task 2.2 (11 points)

Assume arrays  $A1$  and  $A2$ . Each array contains  $n$  distinct integers. Provide the complete C code for procedure `void print_overlap(int A1[], int A2[], int n)`. The procedure prints the integers that appear both in  $A1$  and  $A2$ . You are allowed to use C's `printf` function.

For your solution you are allowed to use the elements that have been declared and defined in the C code fragment at the beginning of this task (`m`, `struct node`, `HT`, `h`, `HTinsert`). You are not allowed to use any other auxiliary composite data structures. For well-chosen hash tables and functions the performance of your solution must be  $O(n)$ .

**plan:** insert all elements of  $A1$  into hash table  $HT$ . lookup each element of  $A2$  in  $HT$  and print the element if the element is present in  $HT$ .

**example:**

- assume  $A1 = [4, 3, 12]$  and  $A2 = [7, 4]$ ;
- for  $m = 4$  we have  $h(7) = 3$ ;  $h(4) = 0$ ;  $h(12) = 0$ ;  $h(3) = 3$
- we insert 3, 4, 12 into  $HT$
- for 7 we search the list that starts at element  $HT[h(7)]$ . the list contains one node with value 3 and we print nothing.
- for 4 we search the list that starts at element  $HT[h(4)]$ . the list contains 2 nodes and one of the nodes has value 4. we print 4.

**code:**

```
1 void print_overlap(int A1[], int A2[], int n){
2     for (int i = 0; i < n; i++) { HTinsert(A1[i]); }
3     for (int i = 0; i < n; i++) {
4         struct node *p = HT[h(A2[i])];
5         while (p != NULL && p->value != A2[i]) { p = p->next; }
6         if (p != NULL) { printf("%d_", A2[i]); }
7     }
8 }
```

### 3 Task 3 (26 points)

Consider a binary tree in which the root node and tree nodes are defined as follows.

```
struct TreeNode {
    int val;
    int aux;
    struct TreeNode* left;
    struct TreeNode* right;
};
```

The field `aux` is an field that you may set, change and use as part of your solutions.

#### 3.1 Task 3.1 (10 points)

Provide *non-recursive* pseudocode for a function `int shortestDist(root)` that gets the root node as the input and returns the shortest distance between the root and a leaf node.

You are **only** allowed to use the abstract data types `Stack` and `Queue` with the following procedures and functions:

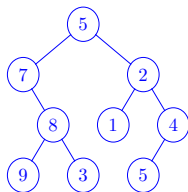
- `struct Stack* initStack()`  
initializes and returns a stack
- `void push(struct Stack* S, void* p)`  
pushes pointer  $p$  onto stack  $S$
- `void* pop(struct Stack* S)`  
removes and returns the top item from a non-empty stack  $S$
- `bool isEmpty(S)`  
returns True if stack  $S$  is empty and False otherwise
- `struct Queue* initQueue()`  
initializes and returns a FIFO queue
- `void enqueue(struct Queue* Q, void* p)`  
add pointer  $p$  at the end of  $Q$
- `void* dequeue(struct Queue* Q)`  
removes and returns the first item from a non-empty queue  $Q$
- `bool isEmpty(Q)`  
returns True if stack  $Q$  is empty and False otherwise

Note: a pointer `void*` is compatible with a pointer of any type.

solution 1:

**plan:** use BFS algorithm with queue Q. Q stores pointers to tree nodes. expand first node of Q by adding neighbors at end of Q. the aux field stores the distance of a node to the root. the distance of the first leaf that is dequeued is the shortest distance between the root and a leaf.

**example:**



$Q = [5 - 0]$   
 $= [7 - 1, 2 - 1]$   
 $= [2 - 1, 8 - 2]$   
 $= [8 - 2, 1 - 2, 4 - 2]$   
 $= [1 - 2, 4 - 2, 9 - 3, 3 - 3]$

**code:**

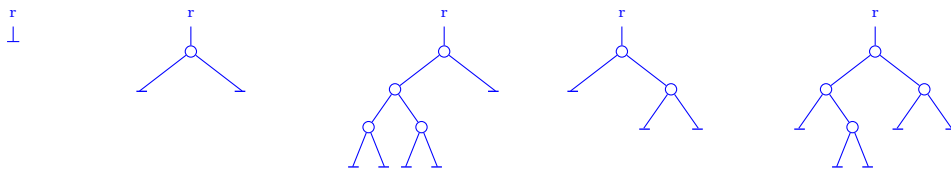
```
1  int shortestDist(struct node* p) {
2      struct Queue* q = initQueue();
3      p->aux = 0; enqueue(q,p);
4      while (!isEmpty(q)) {
5          p = dequeue(q);
6          if (p->l == p->r) { return p->aux; }
7          if (p->l != NULL) { p->l->aux = p->aux + 1; enqueue(q,p->l); }
8          if (p->r != NULL) { p->r->aux = p->aux + 1; enqueue(q,p->r); }
9      }
10 }
```

### 3.2 Task 3.2 (12 points)

Provide *recursive* pseudocode for a function `int shortestDist(root)` that takes the root node as the input and returns the shortest distance between the root and a leaf node. Use the substitution method to prove an upper bound for the runtime complexity of your algorithm.

**plan:** The recursive algorithm considers five cases: node without children, node with only left child, node with only right child, node with two children, and empty tree. for each case shortest distance computation is specified recursively.

**example:** the following trees illustrate the five cases that are considered for node `r` (the first case only happens for an empty tree, i.e., `r == NULL`):



code:

```
1 int shortestDist(struct node* p) {  
2     if (p == NULL) { return -1; }  
3     else if (p->a == p->b) { return 0; }  
4     else if (p->a == NULL) { return shortestDist(p->b) + 1; }  
5     else if (p->b == NULL) { return shortestDist(p->a) + 1; }  
6     else { return min(shortestDist(p->a), shortestDist(p->b)) + 1; }  
7 }
```

1. The runtime complexity is given by the number of calls of `shortestDist`. (All other constructs in the code are constant time.)

2. Line 4 and 5 are special cases of line 6 with one less call of `shortestDist`. Thus, we prove the upper bound for a slower algorithm. The upper bound of the slower algorithm is also an upper bound for the faster algorithm. We have for each non-leaf node two calls of `shortestDist`.

3. The root of a binary tree splits a binary tree with  $n$  nodes into a left subtree  $A$  with  $k$  nodes and a right subtree  $B$  with  $n - k - 1$  nodes.  $k$  is a value between 0 and  $n - 1$  since either  $A$  or  $B$  can be empty. This can be described by the following recurrence ( $N(k)$  is the number of calls of `shortestDist` in a tree with  $k$  nodes):

$$N(n) = N(k) + N(n - k - 1) + 1 \quad k \in [0, n - 1]$$

4. Our guess is a linear complexity since each node is visited once during the traversal. Thus, our guess is  $T(n) = O(n) \leq cn$ .

$$\begin{aligned} N(n) &= N(k) + N(n - k - 1) + 1, k \in [0, n - 1] \\ N(n) &\leq ck + c(n - k - 1) + 1 \\ &= cn - (c - 1) \\ &\leq cn \text{ if } c \geq 1 \end{aligned}$$

5. The base cases are proven as follows:

- $n = 1$ : we call `shortestDist` with the root node and terminate immediately (line 3) since both children are NULL; thus 1 call of `shortestDist`:  $N(1) = 1$
- $n = 2$ : we have the initial call and calls for the left and right subtree; the calls for the subtrees terminate since in one case the subtree is NULL and in the other case both children of the subtree root are NULL; thus 3 calls of `shortestDist`:  $N(2) = 3$
- For  $n = 1$  and  $n = 2$  it therefore holds that  $N(n) < 2n$ .

### 3.3 Task 3.3 (4 points)

Assume the result of calling function `shortestDist` is  $k$ . Determine and explain the number of tree nodes that have been traversed during the execution of `shortestDist`.

The number of nodes depends on the algorithm that is being used. The following are possible algorithms.

case 1: `shortestDist` is implemented with a queue:

- minimum: the number of traversed tree nodes is  $k$  if the binary tree degenerates to a list (in this case  $k$  is equal to  $n$ , i.e., the number of nodes in the tree)
- maximum:
  - the number of dequeued tree nodes is  $2^{k+1} - 1$  if the binary tree is a nearly complete binary tree where the rightmost node at depth  $k$  is a leaf.
  - the number of enqueued tree nodes is  $2^{k+2} - 3$  if the binary tree is a nearly complete binary tree where the rightmost node at depth  $k$  is a leaf.

case 2: `shortestDist` is implemented with a stack or as a recursive algorithm. in this case all nodes are visited once and the number of traversed nodes is  $n$  (number of nodes in the binary tree).

## 4 Task 4 (27 points)

Consider an  $m \times n$  matrix  $M$  that contains integers. The task is to compute the distance from every element of  $M$  to its nearest zero in  $M$ . The distance between positions  $[i, j]$  and  $[p, q]$  is defined as  $|p - i| + |q - j|$ .

The idea for an efficient solution that is proportional to the number of elements in  $M$  is to compute a solution matrix  $S$  with the same dimensions as matrix  $M$ . The value of element  $S[i, j]$  is the distance from position  $[i, j]$  to the nearest zero in  $M$ . Figure 1 shows an example with a  $3 \times 3$  matrix  $M_0$  and corresponding solution matrix  $S_0$ .

|   |    |    |    |
|---|----|----|----|
|   | 0  | 1  | 2  |
| 0 | 0  | -1 | 6  |
| 1 | -3 | -2 | -1 |
| 2 | -5 | -1 | 0  |

$M_0$

|   |   |   |   |
|---|---|---|---|
|   | 0 | 1 | 2 |
| 0 | 0 | 1 | 2 |
| 1 | 1 | 2 | 1 |
| 2 | 2 | 1 | 0 |

$S_0$

Figure 1: Illustration of input matrix  $M$  and solution matrix  $S$

Hint: Instead of directly computing the shortest distance to a zero in all directions it is possible to compute the shortest distance to a zero in some directions only. For instance, a first step can be to compute the shortest distance of an element to a zero in the north (including the cell itself). This distance can then be used to compute the shortest distances to a zero in the north and other directions.

### 4.1 Task 4.1 (3 points)

Consider Figure 2 with a  $4 \times 4$  matrix  $M$ . Complete the corresponding solution matrix  $S$  with the shortest distance to zeros in all directions and the solution matrix  $nS$  with the shortest distances to zeros in the north (including the cell itself).

|    |    |    |    |
|----|----|----|----|
| -1 | 2  | 0  | 1  |
| 1  | -1 | -1 | -1 |
| 1  | 2  | -1 | 1  |
| 0  | 4  | -1 | 0  |

$M$

|   |   |   |   |
|---|---|---|---|
| 2 | 1 | 0 | 1 |
| 2 | 2 | 1 | 2 |
| 1 | 2 | 2 | 1 |
| 0 | 1 | 1 | 0 |

$S$

|   |   |   |   |
|---|---|---|---|
| 8 | 8 | 0 | 8 |
| 8 | 8 | 1 | 8 |
| 8 | 8 | 2 | 8 |
| 0 | 8 | 3 | 0 |

$nS$

Figure 2: Input matrix  $M$  and solution matrix  $S$

## 4.2 Task 4.2 (10 points)

Assume matrix  $M$  is given. State recurrences for

- $nD(i, j)$ : distance to the nearest zero in the north (including the cell itself).
- $D(i, j)$ : distance to the nearest zero (including the cell itself).

Let  $0 \leq i < m$  and  $0 \leq j < n$ .

shortest distances to zeros in the north:

$$nD(i, j) = \begin{cases} 0 & \text{if } M[i, j] = 0 \\ m + n & \text{if } M[i, j] \neq 0 \wedge (i = 0 \vee nD(i - 1, j) = n + m) \\ nD(i - 1, j) + 1 & \text{otherwise} \end{cases}$$

shortest distances to zeros in all directions:

$$nD(i, j) = \begin{cases} 0 & \text{if } M[i, j] = 0 \\ m + n & \text{if } M[i, j] \neq 0 \wedge (i = 0 \vee nD(i - 1, j) = n + m) \\ nD(i - 1, j) + 1 & \text{otherwise} \end{cases}$$

$$snD(i, j) = \begin{cases} nD(i, j) & \text{if } i = m - 1 \\ \min(nD(i, j), snD(i + 1, j) + 1) & \text{otherwise} \end{cases}$$

$$wsnD(i, j) = \begin{cases} snD(i, j) & \text{if } j = 0 \\ \min(snD(i, j), wsnD(i, j - 1) + 1) & \text{otherwise} \end{cases}$$

$$D(i, j) = ewsnD(i, j) = \begin{cases} wsnD(i, j) & \text{if } j = n - 1 \\ \min(wsnD(i, j), ewsnD(i, j + 1) + 1) & \text{otherwise} \end{cases}$$

$D(i, j)$  can be stated directly as:

$$D(i, j) = \begin{cases} 0 & \text{if } M[i, j] = 0 \\ m + n & \text{if } (i < 0) \vee (i \geq m) \vee (j < 0) \vee (j \geq n) \\ \min(\min(D(i - 1, j), D(i, j - 1)), \min(D(i + 1, j), D(i, j + 1))) + 1 & \text{otherwise} \end{cases}$$

### 4.3 Task 4.3 (14 points)

Write a function

- `nD(int i, int j)` that computes the distance of  $M[i,j]$  to the nearest zero in a cell of  $M$  in the north (including  $M[i,j]$ ).
- `D(int i, int j)` that computes the distance of element  $M[i,j]$  to the nearest zero in a cell of  $M$  (including  $M[i,j]$ ).

The time complexity of your solutions must be  $O(m \times n)$ .

dynamic programming solution for shortest distance to zeroes in the north:

```
1 void nD() {
2     for (int i = 0; i < m; i++) {
3         for (int j = 0; j < n; j++) {
4             if (M[i][j] == 0) { S[i][j] = 0; }
5             else if (i == 0) { S[i][j] = m + n; }
6             else { S[i][j] = S[i-1][j] + 1; }
7         }
8     }
```

dynamic programming solution for shortest distance to zeroes in all directions:

```
1 void ewsnd() {
2     for (int i = 0; i < m; i++) {
3         for (int j = 0; j < n; j++) {
4             if (M[i][j] == 0) { S[i][j] = 0; }
5             else if (i == 0 || S[i-1][j] == n + m) { S[i][j] = m + n; }
6             else { S[i][j] = S[i-1][j] + 1; }
7         }
8     }
9     for (int i = m-2; i >= 0; i--) {
10        for (int j = 0; j < n; j++) {
11            S[i][j] = min(S[i][j], S[i+1][j]+1);
12        }
13    }
14    for (int j = 1; j < n; j++) {
15        for (int i = 0; i < m; i++) {
16            S[i][j] = min(S[i][j], S[i][j-1]+1);
17        }
18    }
19    for (int j = m-2; j >= 0; j--) {
20        for (int i = 0; i < m; i++) {
21            S[i][j] = min(S[i][j], S[i][j+1]+1);
22        }
23    }
24 }
```